Vehicle Service Management System on Neo4j

Database Management Systems: Design Assignment 3 (DA3)

Vellore Institute of Technology, Chennai

Reg. No.	Name
25BCE1890	Danda Nanditha
25BCE5102	Pooja KR
25BCE5658	Manasa LS

Repository: https://github.com/NandithaDanda/DBMS-DA3-Neo4j-Vehicle-Service

Database: Neo4j AuraDB Free, queried with Cypher

Abstract

This report documents the re-implementation of the Vehicle Service Management System as a property graph in Neo4j. The system was designed in DA1 and implemented relationally in MySQL in DA2. In DA3 the same domain, comprising customers, vehicles, mechanics, service cards, spare parts, suppliers, invoices and payments, is modelled as nodes and typed relationships and queried with Cypher. The report describes the graph model, the integrity constraints, the sample dataset, the CRUD operations, the ten business queries carried over from DA2 (Q1 to Q10) and nine graph-oriented queries (G1 to G9). It also covers an atomic stock update, an audit log, an index with query-plan inspection, and full-text search. The relational and graph approaches are then compared, with SQL and Cypher versions of selected queries presented side by side.

Contents

1. Introduction and recap of DA1 and DA2
2. Objectives
3. Why Neo4j?
4. Tools and technologies
5. Graph model
6. Constraints and indexes
7. Dataset creation
8. CRUD operations
9. Business queries Q1 to Q10
10. Graph queries G1 to G9
11. Transaction, audit log, index and full-text search

12. MySQL vs Neo4j
13. SQL and Cypher side by side
14. Conclusion
15. References

---

1. Introduction and recap of DA1 and DA2

A vehicle service centre must maintain records of its customers and their vehicles, the service requests raised against those vehicles, the mechanics assigned to each job, the spare parts consumed, the suppliers of those parts, and the invoices and payments that conclude each job. Each of these records is simple in isolation. The value of the system lies in the connections between them.

DA1 was the design stage. The problem domain was selected, the entities and their attributes were identified (customers, vehicles, mechanics, service cards, spare parts, suppliers, invoices and payments), and an EER diagram was prepared. Two features of that diagram are significant for this report. Mechanics may be engine specialists, electrical specialists, both, or neither, so the specialisation is overlapping. Vehicles are either cars or motorcycles.

DA2 implemented the design in MySQL. Entities became tables and relationships became foreign keys. Because spare parts and suppliers are related many-to-many, a junction table (SUPPLIES) was required. The ten business queries (Q1 to Q10) were written in SQL. These queries were satisfactory, but any question that began at one end of the service chain and required information from the other end needed a long sequence of joins.

DA3 retains the business problem and changes the database model. The system is rebuilt as a property graph in Neo4j, in which entities are nodes, the associations between them are typed relationships, and queries are written in Cypher. As the domain is unchanged, Q1 to Q10 could be translated directly, allowing the two models to be compared on identical questions. Nine further graph-oriented queries (G1 to G9) were added, together with features not used in DA2: constraints, an index with query-plan inspection, an audit log, an atomic stock update and full-text search.

The implementation consists of six Cypher scripts, executed in sequence. Each section of this report identifies the script from which its material is drawn.

---

2. Objectives

The objectives of DA3 were as follows.

- To remodel the DA1/DA2 system as a graph, identifying which concepts become nodes, which become relationships and which become labels.
- To preserve the earlier design decisions: overlapping mechanic specialisations, car and motorcycle subtypes, split payments, and the many-to-many association between

parts and suppliers.
- To protect key identifiers with uniqueness constraints, including vehicle registration and chassis numbers.
- To populate the graph with a dataset large enough for every query to return meaningful results.
- To demonstrate create, read, update and delete operations, and the rejection of invalid data by a constraint.
- To translate Q1 to Q10 into Cypher.
- To develop G1 to G9, which depend on traversing relationships across several hops, including recall tracing, supplier impact analysis and shortest-path search.
- To demonstrate an atomic stock update, an audit log, an index with `PROFILE`, and full-text search.
- To compare Neo4j with MySQL objectively, including the cases in which MySQL remains the better choice.
- To maintain all scripts and documentation in a GitHub repository so that the work can be reproduced.

---

3. Why Neo4j?

Reasons for choosing Neo4j

1. The domain is naturally a graph. The chain Customer → Vehicle → Service Card → Service Item → Spare Part → Supplier is a sequence of relationships. The DA1 EER diagram maps onto a graph almost one to one, with entities becoming nodes and relationships becoming edges.

2. Multi-hop queries do not require joins. Identifying the customers affected by a faulty brake pad requires approximately six joins in MySQL once the subtype and supplier tables are included. In Neo4j the same question is expressed as a single path pattern, written in the order in which the question would be asked. Relationships are stored as direct references between nodes, so traversal cost depends on the portion of the graph visited rather than on the total size of the database.

3. Many-to-many relationships are simpler. In SQL, parts and suppliers require a SUPPLIES junction table. In the graph, a part has a `SUPPLIED_BY` relationship to each of its suppliers and no intermediate structure is needed.

4. Specialisation is represented through labels. A mechanic who is an engine specialist, an electrical specialist, both, or neither is a single node carrying zero, one or two additional labels. Cars and motorcycles are handled in the same way. No subtype tables are required.

5. Relationship-based queries are a core strength of Cypher. Shortest-path search, service-history traversal, supplier impact analysis and identification of parts used together

are concise in Cypher but cumbersome in SQL, where they are often implemented with recursion or many joins.

6. The schema is flexible and results can be visualised. New properties or relationship types, such as warranty or customer feedback, can be added without an `ALTER TABLE` statement. The Neo4j graph view also renders query results visually, which assisted in verifying that each query behaved as intended.

Limitations of Neo4j

1. It is not the most suitable choice for every workload. Much of the work of a service centre is tabular, including monthly totals, stock sheets and payroll. These are bulk aggregations over uniform rows, which relational databases handle well. Migrating such reporting to a graph database would offer little benefit.

2. Integrity enforcement is weaker than in a relational database. Neo4j has no foreign keys, so nothing prevents a `ServiceCard` from existing without a vehicle, or a `Payment` without an invoice, unless the application enforces this. Uniqueness constraints are available in all editions, but constraints requiring a property to exist are, according to the Neo4j documentation, limited to Enterprise Edition. The `StatusLog` nodes in this project, which carry no constraint, illustrate the gap.

3. Operational and design considerations remain. Traversals without an upper bound can become expensive, and indexes must still be chosen deliberately. AuraDB Free is adequate for coursework but is subject to size limits, and a production deployment would need to address backup, access control and cost. In addition, most reporting tools assume SQL, so integrating a graph database with them requires additional effort.

---

4. Tools and technologies

Tool	Purpose
Neo4j AuraDB Free	The cloud-hosted graph database that stores everything
Cypher	Constraints, data loading, CRUD, queries, traversals
Neo4j Query tool / Workspace	Executing Cypher, viewing results as tables or graphs, capturing screenshots
MySQL and SQL	The DA2 relational baseline used for comparison
GitHub	Scripts, screenshots and this report
Markdown	Format of this report, so it renders directly on GitHub

Repository layout

```
scripts/
  01_constraints.cypher
  02_load_data.cypher
  03_crud.cypher
  04_queries_Q1_Q10.cypher
  05_graph_queries_G1_G9.cypher
  06_extras.cypher
screenshots/
report/
README.md
```

The scripts should be run in this order on an empty database. The loader uses `CREATE`, so a second run would duplicate data and violate the uniqueness constraints. One statement in the CRUD script is intended to fail (section 8.4).

---

5. Graph model

5.1 Design approach

Concepts with their own identity and properties, such as a customer, a service card or an invoice, were modelled as nodes. Concepts that describe how two nodes are related, such as ownership, usage or supply, were modelled as relationships. Subtypes (car, motorcycle, engine mechanic, electrical mechanic, part item and labour item) were modelled as additional labels on the same node rather than as separate nodes.

The main spine of the graph is:

```
Customer → Vehicle → ServiceCard → ServiceItem → SparePart → Supplier
```

with mechanics attached to service cards, and invoices, payments and status logs hanging off the service card side.

5.2 Node table

Node label	Key property	Other properties	Purpose
Customer	person_id	name, phone, address, registration_date	A person who brings a vehicle in
Mechanic	mechanic_id	name, phone, joining_date, salary, shift	Staff who work on jobs
Vehicle	vehicle_id	chassis_no, registration_no, brand	Common identity of any vehicle
Car	(extra label)	none	Marks a Vehicle as a car
Motorcycle	(extra label)	none	Marks a Vehicle as a motorcycle
EngineMechanic	(extra label)	none	Marks a Mechanic as an engine specialist
ElectricalMechanic	(extra label)	none	Marks a Mechanic as an electrical specialist
ServiceCard	service_id	start_date, end_date, complaint, est_delivery, actual_delivery, status	One service job
ServiceItem	item_id	quantity, unit_price, discount	One line on a service card
PartUsed	(extra label on ServiceItem)	none	Line item that consumes a spare part
LabourService	(extra label on ServiceItem)	labour_hours	Line item that charges labour
SparePart	part_id	part_name, stock_quantity, unit_price	Catalogue and stock
Supplier	supplier_id	name, phone, address	Who supplies parts
Invoice	invoice_id	invoice_date, subtotal, tax, total	The bill for a service card

Node label	Key property	Other properties	Purpose
Payment	payment_id	payment_date, amount, method, upi_id, txn_id, card_last4, bank	One payment against an invoice
StatusLog	none	old_status, new_status, changed_at	One recorded status change

A node with two labels remains a single node. A car in the dataset is one node labelled `Vehicle:Car`, so label counts from a verification query exceed the number of nodes.

5.3 Relationship table

Relationship	Direction	Meaning
OWNS	Customer → Vehicle	The customer owns the vehicle
HAS_SERVICE	Vehicle → ServiceCard	The vehicle has this service job
ASSIGNED_TO	Mechanic → ServiceCard	The mechanic is working on this job
INCLUDES	ServiceCard → ServiceItem	The job contains this line item
USES	PartUsed → SparePart	The line item uses this part
SUPPLIED_BY	SparePart → Supplier	The supplier can provide this part
GENERATES	ServiceCard → Invoice	The job produced this invoice
PAYS	Payment → Invoice	The payment goes towards this invoice
HAS_LOG	ServiceCard → StatusLog	The job has this status-change record

5.4 Schema screenshot

The data loading script ends with `CALL db.schema.visualization();`, which displays the labels and relationship types present in the database.

Schema visualisation

Figure 1. Schema of the Vehicle Service graph.

5.5 A service job in the graph

Consider the first visit of the customer Arun Kumar. Arun is a `Customer` node and `OWNS` a Honda car. The car `HAS_SERVICE` service card 1, to which the mechanic Suresh B is `ASSIGNED_TO`. The card `INCLUDES` two line items: a `PartUsed` item that `USES` an Oil Filter (`SUPPLIED_BY` AutoParts Co) and a `LabourService` item. The card `GENERATES` invoice 1,

which a `Payment` `PAYS`. Each step in this description corresponds to one relationship in the database.

---

6. Constraints and indexes

6.1 Uniqueness constraints (`01_constraints.cypher`)

```cypher
CREATE CONSTRAINT cust_id IF NOT EXISTS FOR (c:Customer)    REQUIRE c.person_id
CREATE CONSTRAINT mech_id IF NOT EXISTS FOR (m:Mechanic)    REQUIRE m.mechanic_
CREATE CONSTRAINT veh_id  IF NOT EXISTS FOR (v:Vehicle)     REQUIRE v.vehicle_i
CREATE CONSTRAINT veh_reg IF NOT EXISTS FOR (v:Vehicle)     REQUIRE v.registrat
CREATE CONSTRAINT veh_chs IF NOT EXISTS FOR (v:Vehicle)     REQUIRE v.chassis_n
CREATE CONSTRAINT svc_id  IF NOT EXISTS FOR (s:ServiceCard) REQUIRE s.service_i
CREATE CONSTRAINT item_id IF NOT EXISTS FOR (i:ServiceItem) REQUIRE i.item_id
CREATE CONSTRAINT part_id IF NOT EXISTS FOR (p:SparePart)   REQUIRE p.part_id
CREATE CONSTRAINT sup_id  IF NOT EXISTS FOR (s:Supplier)    REQUIRE s.supplier_
CREATE CONSTRAINT inv_id  IF NOT EXISTS FOR (i:Invoice)     REQUIRE i.invoice_i
CREATE CONSTRAINT pay_id  IF NOT EXISTS FOR (p:Payment)    REQUIRE p.payment_id
```

`IF NOT EXISTS` allows the script to be re-run without errors. There are eleven constraints: one for each main identifier, and two additional constraints on `Vehicle` so that a registration number or chassis number cannot be entered twice. These implement the real-world rule that two vehicles cannot share a registration plate.

A uniqueness constraint checks only values that exist. It does not require a property to be present and does not constrain relationships. The Neo4j documentation also notes that a uniqueness constraint creates a backing index, so lookups by `person_id`, `vehicle_id` and the other keys are already indexed.

Constraints

Figure 2. Constraints created successfully.

6.2 Range index on service status (`06_extras.cypher`)

```cypher
CREATE INDEX svc_status IF NOT EXISTS FOR (s:ServiceCard) ON (s.status);
```

Status is the property most frequently used for filtering (Q2, Q3, G3 and G9), making it the natural candidate for an index. Section 11.3 presents the `PROFILE` output.

6.3 Full-text index on complaints (`06_extras.cypher`)

```cypher
```

```
CREATE FULLTEXT INDEX complaint_search IF NOT EXISTS
FOR (s:ServiceCard) ON EACH [s.complaint];
```

A standard index matches exact values. This index splits the complaint text into terms so that the text can be searched (section 11.4).

---

7. Dataset creation

The dataset is defined in `scripts/02_load_data.cypher`. Each block follows the same pattern: `UNWIND` a list of maps, one per record, followed by `CREATE`, or by `MATCH` and `CREATE` where links to existing nodes are required. Dates are supplied as strings and converted with `date()`. Open jobs have `null` end and delivery dates.

The script creates the following:

Item	Count
Customers	5
Mechanics	3
Vehicles	8 (5 cars, 3 motorcycles)
Spare parts	8
Suppliers	4
Part–supplier links	11
Service cards	10 (5 Completed, 3 In Progress, 2 Received)
Service items	23 (13 part items, 10 labour items)
Invoices	5
Payments	6 (invoice 2 is paid in two parts)

This gives 72 nodes and 86 relationships, counted from the script. Tax on every invoice is 18% of the subtotal (for invoice 1, 1050.00 + 189.00 = 1239.00).

7.1 Customers

cypher

```
UNWIND [
  {id:1,name:'Arun Kumar', phone:'9876501001',addr:'Chennai',   reg:'2026-01-10
  {id:2,name:'Meena Ravi', phone:'9876501002',addr:'Tambaram',  reg:'2026-02-12
  {id:3,name:'Karthik S',  phone:'9876501004',addr:'Adyar',     reg:'2026-03-01
  {id:4,name:'Priya Nair', phone:'9876501005',addr:'Anna Nagar',reg:'2026-03-15
  {id:5,name:'Rahul Verma',phone:'9876501006',addr:'Velachery', reg:'2026-04-02
] AS r
CREATE (:Customer {
  person_id:r.id,
  name:r.name,
  phone:r.phone,
  address:r.addr,
  registration_date:date(r.reg)
});
```

7.2 Mechanics and specialisations

cypher

```
UNWIND [
  {id:1,name:'Suresh B',phone:'9876501003',join:'2024-06-01',sal:28000,shift:'D
  {id:2,name:'Ravi T',  phone:'9876502001',join:'2023-02-15',sal:26000,shift:'N
  {id:3,name:'Divya M', phone:'9876502002',join:'2025-01-20',sal:27000,shift:'D
] AS r
CREATE (:Mechanic {
  mechanic_id:r.id,
  name:r.name,
  phone:r.phone,
  joining_date:date(r.join),
  salary:r.sal,
  shift:r.shift
});

MATCH (m:Mechanic) WHERE m.mechanic_id IN [1,2] SET m:EngineMechanic;
MATCH (m:Mechanic) WHERE m.mechanic_id IN [1,3] SET m:ElectricalMechanic;
```

Suresh (id 1) appears in both lists and therefore carries both labels. Ravi is an engine specialist only and Divya is an electrical specialist only.

7.3 Vehicles

cypher

```
UNWIND [
  {id:1,chs:'CHS10001',reg:'TN01AB1234',brand:'Honda',  owner:1},
  {id:2,chs:'CHS10002',reg:'TN10CD5678',brand:'Hyundai',owner:2},
  {id:4,chs:'CHS10004',reg:'TN22GH3456',brand:'Toyota', owner:3},
  {id:6,chs:'CHS10006',reg:'TN01LM2345',brand:'Maruti', owner:4},
  {id:7,chs:'CHS10007',reg:'TN07NP6789',brand:'Hyundai',owner:5}
] AS r
MATCH (c:Customer {person_id:r.owner})
CREATE (c)-[:OWNS]->(:Vehicle:Car {
  vehicle_id:r.id,
  chassis_no:r.chs,
  registration_no:r.reg,
  brand:r.brand
});

UNWIND [
  {id:3,chs:'CHS10003',reg:'TN01EF9012',brand:'Honda',        owner:1},
  {id:5,chs:'CHS10005',reg:'TN09JK7890',brand:'Yamaha',       owner:4},
  {id:8,chs:'CHS10008',reg:'TN10QR1122',brand:'Royal Enfield',owner:2}
] AS r
MATCH (c:Customer {person_id:r.owner})
CREATE (c)-[:OWNS]->(:Vehicle:Motorcycle {
  vehicle_id:r.id,
  chassis_no:r.chs,
  registration_no:r.reg,
  brand:r.brand
});
```

7.4 Spare parts, suppliers and who supplies what

cypher

```
UNWIND [
  {id:1,name:'Oil Filter',    stock:40,price:250},
  {id:2,name:'Brake Pad',     stock:25,price:900},
  {id:3,name:'Spark Plug',    stock:60,price:150},
  {id:4,name:'Air Filter',    stock:30,price:350},
  {id:5,name:'Battery',       stock:8, price:4500},
  {id:6,name:'Clutch Plate',  stock:6, price:1200},
  {id:7,name:'Headlight Bulb',stock:50,price:120},
  {id:8,name:'Engine Oil 5L', stock:35,price:2100}
] AS r
CREATE (:SparePart {
  part_id:r.id,
  part_name:r.name,
  stock_quantity:r.stock,
  unit_price:r.price
});

UNWIND [
  {id:1,name:'AutoParts Co',    phone:'9000000001',addr:'Guindy'},
  {id:2,name:'SpeedSpares',     phone:'9000000002',addr:'Ambattur'},
  {id:3,name:'PowerCell Traders',phone:'9000000003',addr:'Perungudi'},
  {id:4,name:'MotoMart',        phone:'9000000004',addr:'T Nagar'}
] AS r
CREATE (:Supplier {
  supplier_id:r.id,
  name:r.name,
  phone:r.phone,
  address:r.addr
});

UNWIND [[1,1],[2,1],[2,2],[3,2],[4,1],[5,3],[6,4],[7,3],[7,4],[8,1],[8,2]] AS p
MATCH (p:SparePart {part_id:pair[0]}),
      (s:Supplier  {supplier_id:pair[1]})
CREATE (p)-[:SUPPLIED_BY]->(s);
```

Each pair is `[part_id, supplier_id]`. Brake Pad, Headlight Bulb and Engine Oil each have two suppliers, which provides genuine many-to-many data.

7.5 Service cards and mechanic assignments

cypher

```
UNWIND [
  {id:1, veh:1,mech:1,start:'2026-03-05',end:'2026-03-08',comp:'Engine noise an
  {id:2, veh:2,mech:2,start:'2026-03-10',end:'2026-03-12',comp:'Brake squealing
  {id:3, veh:3,mech:1,start:'2026-03-20',end:'2026-03-21',comp:'Bike not starti
  {id:4, veh:4,mech:2,start:'2026-04-02',end:'2026-04-05',comp:'Regular service
  {id:5, veh:5,mech:3,start:'2026-04-10',end:'2026-04-11',comp:'Headlight not w
  {id:6, veh:1,mech:1,start:'2026-05-02',end:null,        comp:'AC not cooling
  {id:7, veh:6,mech:2,start:'2026-05-05',end:null,        comp:'Clutch slipping
  {id:8, veh:7,mech:1,start:'2026-05-08',end:null,        comp:'Overheating eng
  {id:9, veh:2,mech:2,start:'2026-05-10',end:null,        comp:'Periodic servic
  {id:10,veh:8,mech:3,start:'2026-05-12',end:null,        comp:'Spark plug misf
] AS r
MATCH (v:Vehicle  {vehicle_id:r.veh}),
      (m:Mechanic {mechanic_id:r.mech})
CREATE (v)-[:HAS_SERVICE]->(s:ServiceCard {
  service_id:r.id,
  start_date:date(r.start),
  end_date:date(r.end),
  complaint:r.comp,
  est_delivery:date(r.est),
  actual_delivery:date(r.act),
  status:r.st
})
CREATE (m)-[:ASSIGNED_TO]->(s);
```

7.6 Service items (parts and labour)

cypher

```
UNWIND [
  {item:1, svc:1, part:1,qty:1,price:250},
  {item:3, svc:2, part:2,qty:1,price:900},
  {item:5, svc:3, part:3,qty:1,price:150},
  {item:7, svc:4, part:1,qty:1,price:250},
  {item:8, svc:4, part:8,qty:1,price:2100},
  {item:10,svc:4, part:4,qty:1,price:350},
  {item:11,svc:5, part:7,qty:2,price:120},
  {item:13,svc:6, part:5,qty:1,price:4500},
  {item:15,svc:7, part:6,qty:1,price:1200},
  {item:18,svc:8, part:8,qty:1,price:2100},
  {item:19,svc:9, part:1,qty:1,price:250},
  {item:20,svc:9, part:4,qty:1,price:350},
  {item:22,svc:10,part:3,qty:4,price:150}
] AS r
MATCH (s:ServiceCard {service_id:r.svc}),
      (p:SparePart   {part_id:r.part})
CREATE (s)-[:INCLUDES]->(:ServiceItem:PartUsed {
  item_id:r.item,
  quantity:r.qty,
  unit_price:r.price,
  discount:0
})-[:USES]->(p);

UNWIND [
  {item:2, svc:1, hrs:2,  charge:800},
  {item:4, svc:2, hrs:1.5,charge:600},
  {item:6, svc:3, hrs:1,  charge:400},
  {item:9, svc:4, hrs:1.5,charge:700},
  {item:12,svc:5, hrs:0.5,charge:300},
  {item:14,svc:6, hrs:2,  charge:900},
  {item:16,svc:7, hrs:3,  charge:1500},
  {item:17,svc:8, hrs:2,  charge:1000},
  {item:21,svc:9, hrs:1.5,charge:700},
  {item:23,svc:10,hrs:1,  charge:500}
] AS r
MATCH (s:ServiceCard {service_id:r.svc})
CREATE (s)-[:INCLUDES]->(:ServiceItem:LabourService {
  item_id:r.item,
  quantity:1,
  unit_price:r.charge,
  discount:0,
  labour_hours:r.hrs
});
```

Item identifiers alternate between part and labour items and are unique across both, as required by the `item_id` constraint on `ServiceItem`.

7.7 Invoices and payments

cypher

```
UNWIND [
  {id:1,svc:1,date:'2026-03-08',sub:1050.0,tax:189.0,total:1239.0},
  {id:2,svc:2,date:'2026-03-13',sub:1500.0,tax:270.0,total:1770.0},
  {id:3,svc:3,date:'2026-03-21',sub:550.0, tax:99.0, total:649.0},
  {id:4,svc:4,date:'2026-04-05',sub:3400.0,tax:612.0,total:4012.0},
  {id:5,svc:5,date:'2026-04-11',sub:540.0, tax:97.2, total:637.2}
] AS r
MATCH (s:ServiceCard {service_id:r.svc})
CREATE (s)-[:GENERATES]->(:Invoice {
  invoice_id:r.id,
  invoice_date:date(r.date),
  subtotal:r.sub,
  tax:r.tax,
  total:r.total
});

UNWIND [
  {id:1,inv:1,date:'2026-03-08',amt:1239.0,method:'UPI', upi:'arun@upi', txn:'T
  {id:2,inv:2,date:'2026-03-13',amt:1000.0,method:'Cash',upi:null,       txn:nu
  {id:3,inv:2,date:'2026-03-13',amt:770.0, method:'Card',upi:null,       txn:nu
  {id:4,inv:3,date:'2026-03-21',amt:649.0, method:'Cash',upi:null,       txn:nu
  {id:5,inv:4,date:'2026-04-05',amt:4012.0,method:'Card',upi:null,       txn:nu
  {id:6,inv:5,date:'2026-04-11',amt:637.2, method:'UPI', upi:'priya@upi',txn:'T
] AS r
MATCH (i:Invoice {invoice_id:r.inv})
CREATE (:Payment {
  payment_id:r.id,
  payment_date:date(r.date),
  amount:r.amt,
  method:r.method,
  upi_id:r.upi,
  txn_id:r.txn,
  card_last4:r.last4,
  bank:r.bank
})-[:PAYS]->(i);
```

Payments 2 and 3 together settle invoice 2 (1000 in cash and 770 by card). Properties that do not apply to a payment method are `null`, and Neo4j does not store null properties, so no

empty columns arise as they would in a wide SQL payment table.

7.8 Verification

```cypher
MATCH (n) UNWIND labels(n) AS label
RETURN label, count(*) AS total
ORDER BY label;

CALL db.schema.visualization();
```

Label counts

Figure 3. Node counts per label after loading.

---

8. CRUD operations (`03_crud.cypher`)

8.1 Create

A new customer, her car and an initial service card are created in a single statement.

```cypher
CREATE (c:Customer {
  person_id:6,
  name:'Anita Das',
  phone:'9876501007',
  address:'Porur',
  registration_date:date()
})
CREATE (c)-[:OWNS]->(v:Vehicle:Car {
  vehicle_id:9,
  chassis_no:'CHS10009',
  registration_no:'TN11ST4455',
  brand:'Tata'
})
CREATE (v)-[:HAS_SERVICE]->(s:ServiceCard {
  service_id:11,
  start_date:date(),
  complaint:'Steering vibration',
  est_delivery:date()+duration({days:3}),
  status:'Received'
})
RETURN c, v, s;
```

Create

8.2 Read

```cypher
MATCH (c:Customer {name:'Anita Das'})-[:OWNS]->(v)-[:HAS_SERVICE]->(s)
RETURN c.name, v.registration_no, s.service_id, s.complaint, s.status;
```

One row: Anita Das, TN11ST4455, service 11, "Steering vibration", Received.

Read

8.3 Update

The job is assigned to Divya and its status is changed to In Progress.

```cypher
MATCH (s:ServiceCard {service_id:11}),
      (m:Mechanic    {mechanic_id:3})
CREATE (m)-[:ASSIGNED_TO]->(s)
SET s.status = 'In Progress'
RETURN s.service_id, s.status, m.name;
```

Update

8.4 Constraint check (expected to fail)

```cypher
CREATE (:Vehicle {
  vehicle_id:99,
  chassis_no:'CHSXXXX',
  registration_no:'TN01AB1234',
  brand:'Test'
});
```

The registration number `TN01AB1234` already belongs to the Honda car owned by Arun Kumar, so the `veh_reg` constraint rejects the statement and nothing is written. The error message shown in the screenshot confirms that the constraint is active.

Duplicate rejected

8.5 Delete and verify

```cypher
```

```
MATCH (c:Customer {person_id:6})-[:OWNS]->(v:Vehicle)
OPTIONAL MATCH (v)-[:HAS_SERVICE]->(s:ServiceCard)
DETACH DELETE s, v, c;

MATCH (c:Customer {person_id:6}) RETURN c;
```

`DETACH DELETE` removes a node together with every relationship attached to it; a plain `DELETE` is refused if the node still has relationships. The second query returns zero rows, confirming that the test customer has been removed and the dataset is restored.

Delete verified

---

9. Business queries Q1 to Q10 (`04_queries_Q1_Q10.cypher`)

The results below correspond to the dataset in section 7.

Q1. Vehicles with owner details

cypher

```
MATCH (c:Customer)-[:OWNS]->(v:Vehicle)
RETURN v.vehicle_id, v.registration_no, v.brand,
       c.name AS customer, c.phone
ORDER BY c.name;
```

8 rows, one per vehicle, grouped by owner.

Q1

Q2. Service cards that are not completed

cypher

```
MATCH (s:ServiceCard)
WHERE s.status <> 'Completed'
RETURN s.service_id, s.complaint, s.status, s.est_delivery
ORDER BY s.est_delivery;
```

Service cards 6, 7, 8, 9 and 10, in that order of estimated delivery.

Q2

Q3. Jobs by status

cypher

```
MATCH (s:ServiceCard)
RETURN s.status, count(*) AS total_jobs
ORDER BY total_jobs DESC;
```

status	total_jobs
Completed	5
In Progress	3
Received	2

Q3

Q4. Mechanics with more than two jobs

```cypher
MATCH (m:Mechanic)-[:ASSIGNED_TO]->(s:ServiceCard)
WITH m, count(s) AS jobs
WHERE jobs > 2
RETURN m.name, jobs;
```

Suresh B and Ravi T, with four jobs each. Divya M has only two jobs and is therefore excluded.

Q4

Q5. Honda vehicles

```cypher
MATCH (v:Vehicle {brand:'Honda'})
RETURN v.registration_no, v.brand, labels(v) AS type;
```

Two rows: `TN01AB1234` (Vehicle, Car) and `TN01EF9012` (Vehicle, Motorcycle). `labels(v)` shows the subtype without needing a separate table.

Q5

Q6. Registration numbers containing TN01

```cypher
```

```
MATCH (v:Vehicle)
WHERE v.registration_no CONTAINS 'TN01'
RETURN v.registration_no, v.brand;
```

Three vehicles: `TN01AB1234`, `TN01EF9012` and `TN01LM2345`.

Q6

Q7. Total value of spare-part stock

```cypher
MATCH (p:SparePart)
RETURN sum(p.stock_quantity * p.unit_price) AS total_stock_value;
```

174700. Engine oil alone is 35 × 2100 = 73,500, about 42% of the total.

Q7

Q8. Three highest invoices

```cypher
MATCH (i:Invoice)
RETURN i.invoice_id, i.total
ORDER BY i.total DESC
LIMIT 3;
```

invoice_id	total
4	4012.0
2	1770.0
1	1239.0

Q8

Q9. Customers with more than one vehicle

```cypher
MATCH (c:Customer)-[:OWNS]->(v:Vehicle)
WITH c, count(v) AS vehicles
WHERE vehicles > 1
RETURN c.name, vehicles;
```

Arun Kumar, Meena Ravi and Priya Nair, with two vehicles each. Karthik S and Rahul Verma have one each.

Q9

Q10. Payments received per invoice

```cypher
MATCH (i:Invoice)<-[:PAYS]-(p:Payment)
RETURN i.invoice_id, i.total AS invoice_total,
       sum(p.amount) AS paid, count(p) AS payments
ORDER BY i.invoice_id;
```

invoice_id	invoice_total	paid	payments
1	1239.0	1239.0	1
2	1770.0	1770.0	2
3	649.0	649.0	1
4	4012.0	4012.0	1
5	637.2	637.2	1

Every invoice is fully paid, and invoice 2 demonstrates a split payment.

Q10

---

10. Graph queries G1 to G9 (`05_graph_queries_G1_G9.cypher`)

These queries would have been awkward to express in the DA2 relational model. G1, G4 and G7 are best viewed in the graph result view rather than as tables.

G1. A customer's invoiced service history

```cypher
MATCH path =
  (c:Customer {name:'Arun Kumar'})-[:OWNS]->
  (:Vehicle)-[:HAS_SERVICE]->(:ServiceCard)-[:GENERATES]->(:Invoice)
RETURN path;
```

Two paths come back: the Honda car's service 1 with invoice 1, and the Honda motorcycle's service 3 with invoice 3. Service 6 is absent because that job is still open and has no invoice.

G1

G2. Everything about one vehicle

```cypher
MATCH (v:Vehicle {registration_no:'TN01AB1234'})-[:HAS_SERVICE]->(s:ServiceCard
OPTIONAL MATCH (m:Mechanic)-[:ASSIGNED_TO]->(s)
OPTIONAL MATCH (s)-[:INCLUDES]->(:PartUsed)-[:USES]->(p:SparePart)-[:SUPPLIED_B
RETURN s.service_id, s.status, m.name AS mechanic,
       p.part_name, sup.name AS supplier;
```

service_id	status	mechanic	part_name	supplier
1	Completed	Suresh B	Oil Filter	AutoParts Co
6	In Progress	Suresh B	Battery	PowerCell Traders

`OPTIONAL MATCH` retains a service card in the result even when no mechanic, part or supplier is yet attached to it.

G2

G3. Which open jobs depend on one supplier

```cypher
MATCH (sup:Supplier {name:'AutoParts Co'})<-[:SUPPLIED_BY]-
      (p:SparePart)<-[:USES]-(:PartUsed)<-[:INCLUDES]-
      (s:ServiceCard)
WHERE s.status <> 'Completed'
RETURN DISTINCT s.service_id, s.status, p.part_name;
```

Three rows: service 8 (Engine Oil 5L, Received) and service 9 (Oil Filter and Air Filter, In Progress). If AutoParts Co were unable to deliver, these two jobs would be delayed.

G3

G4. Recall analysis for the Oil Filter

Graph view:

```cypher
```

```
MATCH path =
  (p:SparePart {part_name:'Oil Filter'})<-[:USES]-
  (:PartUsed)<-[:INCLUDES]-(:ServiceCard)<-[:HAS_SERVICE]-
  (v:Vehicle)<-[:OWNS]-(c:Customer)
RETURN path;
```

Table view:

cypher

```
MATCH (p:SparePart {part_name:'Oil Filter'})<-[:USES]-
      (:PartUsed)<-[:INCLUDES]-(:ServiceCard)<-[:HAS_SERVICE]-
      (v:Vehicle)<-[:OWNS]-(c:Customer)
RETURN DISTINCT c.name, c.phone, v.registration_no;
```

name	phone	registration_no
Arun Kumar	9876501001	TN01AB1234
Karthik S	9876501004	TN22GH3456
Meena Ravi	9876501002	TN10CD5678

If a batch of oil filters were found to be faulty, this result is the list of customers to contact. Of all the queries, this one demonstrates the value of the graph model most clearly.

G4

G5. Parts used together

cypher

```
MATCH (p1:SparePart)<-[:USES]-(:PartUsed)<-[:INCLUDES]-
      (s:ServiceCard)-[:INCLUDES]->(:PartUsed)-[:USES]->(p2:SparePart)
WHERE p1.part_id < p2.part_id
RETURN p1.part_name, p2.part_name,
       count(DISTINCT s) AS times_together
ORDER BY times_together DESC;
```

p1	p2	times_together
Oil Filter	Air Filter	2
Oil Filter	Engine Oil 5L	1
Air Filter	Engine Oil 5L	1

The condition `p1.part_id < p2.part_id` prevents each pair from appearing twice in opposite orders. The oil filter and air filter occur together on both routine services (4 and 9). Ten jobs is a very small sample, but the approach is the basis for recommending service packages.

G5

G6. Mechanics with both specialisations

```cypher
MATCH (m:Mechanic:EngineMechanic:ElectricalMechanic)
RETURN m.name, labels(m) AS roles;
```

Only Suresh B, with `Mechanic`, `EngineMechanic` and `ElectricalMechanic` in his labels.

G6

G7. Shortest path from a customer to a supplier

```cypher
MATCH (c:Customer {name:'Arun Kumar'}),
      (s:Supplier  {name:'AutoParts Co'})
MATCH path = shortestPath((c)-[*..10]-(s))
RETURN path;
```

The path contains five relationships: Arun Kumar → his Honda car → service card 1 → the oil filter line item → Oil Filter → AutoParts Co. The pattern `-[*..10]-` is undirected, so edges may be traversed in either direction, and `..10` limits the search depth.

In SQL this would require a recursive query over several tables; in Cypher it is a single function call.

G7

G8. Revenue per mechanic

```cypher
```

```
MATCH (m:Mechanic)-[:ASSIGNED_TO]->(:ServiceCard)-[:GENERATES]->(i:Invoice)
RETURN m.name, count(i) AS invoices, sum(i.total) AS revenue
ORDER BY revenue DESC;
```

name	invoices	revenue
Ravi T	2	5782.0
Suresh B	2	1888.0
Divya M	1	637.2

This figure is billed revenue only. It does not account for the cost of parts or labour and is therefore not profit.

G8

G9. Low-stock parts needed by pending jobs

cypher

```
MATCH (s:ServiceCard)-[:INCLUDES]->(:PartUsed)-[:USES]->(p:SparePart)
WHERE s.status IN ['In Progress','Received']
  AND p.stock_quantity < 10
RETURN DISTINCT p.part_name, p.stock_quantity, s.service_id;
```

part_name	stock_quantity	service_id
Battery	8	6
Clutch Plate	6	7

Both parts are required by jobs that are still open, and both have fewer than ten units in stock.

G9

---

11. Transaction, audit log, index and full-text search (`06_extras.cypher`)

11.1 Atomic stock update (D1)

Adding two headlight bulbs to service card 10 must accomplish two things together: creating the line item and deducting two units from stock. Completing only one would leave the records inconsistent.

cypher

```
MATCH (s:ServiceCard {service_id:10}),
      (p:SparePart   {part_id:7})
WHERE p.stock_quantity >= 2
CREATE (s)-[:INCLUDES]->(:ServiceItem:PartUsed {
  item_id:24,
  quantity:2,
  unit_price:p.unit_price,
  discount:0
})-[:USES]->(p)
SET p.stock_quantity = p.stock_quantity - 2
RETURN p.part_name, p.stock_quantity;
```

A single Cypher statement runs as one transaction, so either the item is created and the stock is reduced, or neither occurs. The result shows the Headlight Bulb stock changing from 50 to 48.

The `WHERE` clause acts as a guard. If fewer than two bulbs are in stock, the `MATCH` returns nothing and the statement has no effect and raises no error. A production application would need to report this condition, and concurrent updates would require more care than a single statement provides.

D1

11.2 Audit log (D2)

MySQL supports triggers, whereas Neo4j does not. The change is therefore logged explicitly, with the log node created in the same statement as the status update.

cypher

```
MATCH (s:ServiceCard {service_id:8})
WITH s, s.status AS old_status
SET s.status = 'In Progress'
CREATE (s)-[:HAS_LOG]->(:StatusLog {
  old_status:old_status,
  new_status:'In Progress',
  changed_at:datetime()
})
RETURN s.service_id, old_status, s.status;
```

Reading the log back:

cypher

```
MATCH (s:ServiceCard)-[:HAS_LOG]->(l:StatusLog)
RETURN s.service_id, l.old_status, l.new_status, l.changed_at;
```

Service 8 changes from Received to In Progress, and the change is timestamped. The weakness of this approach is that nothing forces every status change through this statement. If another query sets `status` directly, no log entry is written, so the method depends on consistent application behaviour.

D2

11.3 Index and query plan (D3)

```cypher
CREATE INDEX svc_status IF NOT EXISTS FOR (s:ServiceCard) ON (s.status);

PROFILE
MATCH (s:ServiceCard {status:'In Progress'})
RETURN s.service_id;
```

`PROFILE` executes the query and reports the plan, including the operators used and the rows and database hits for each. With the index in place, the plan is expected to begin with an index seek on `status` rather than a scan of every `ServiceCard`. With only ten service cards there is no meaningful time saving, so the purpose is to demonstrate plan inspection rather than speed.

D3

11.4 Full-text search (D4)

```cypher
CALL db.index.fulltext.queryNodes(
  'complaint_search',
  'brake OR clutch OR engine'
) YIELD node, score
RETURN node.service_id, node.complaint, score
ORDER BY score DESC;
```

Four service cards match: 1 ("Engine noise and oil leak"), 2 ("Brake squealing"), 7 ("Clutch slipping") and 8 ("Overheating engine"). Each comes with a relevance score. The search is case-insensitive and matches whole words within the complaint text, which is difficult to achieve with `=` or `CONTAINS` across several terms. A newly created index takes a short time to populate, so a query run immediately afterwards may return no results.

D4

11.5 Repeating the extras

D1 and D2 modify data. If repeated, D1 would attempt to create a second item 24 (which the constraint rejects) and D2 would write an additional log entry. The script contains commented-out cleanup statements for resetting the data beforehand.

---

12. MySQL vs Neo4j

	MySQL (DA2)	Neo4j (DA3)
Basic structure	Tables, rows, columns	Nodes, relationships, properties
Customer owns a vehicle	Foreign key from Vehicle to Customer	`(:Customer)-[:OWNS]->(:Vehicle)`
Part supplied by many suppliers	Junction table `SUPPLIES`	`SUPPLIED_BY` edges
Linking across entities	JOIN on key columns	Path pattern across relationships
Subtypes (car/motorcycle, mechanic types)	Subtype tables or a type column	Extra labels on the same node
Overlapping specialisation	Needs a separate table or flags	A node can hold several labels
Schema changes	`ALTER TABLE`, often a migration	New properties and relationship types can be added as needed
Integrity	Primary and foreign keys, checks, not-null	Uniqueness constraints; existence constraints only in Enterprise; no foreign keys
Triggers	Built in	Not available; changes are logged within the query
Shortest path / recursion	Recursive CTE	`shortestPath()`
Text search	`FULLTEXT` index (engine dependent)	Full-text index through `db.index.fulltext.queryNodes`
Bulk tabular reports	Very good	Possible, but not what it is built for
Result presentation	Result grids	Result grids and a graph view
Learning curve	Already familiar from DA2	New syntax and a different data-modelling approach

Neither system is superior in every respect. For stock sheets and monthly totals, MySQL remains the more suitable choice. For recall tracing, supplier impact analysis and other

questions that follow a chain of relationships, the Neo4j version was shorter and easier to read.

---

13. SQL and Cypher side by side

The SQL below assumes the DA2 table layout, with the same key columns as the graph properties.

Example 1. Customers with more than one vehicle (Q9)

SQL

```sql
SELECT c.name, COUNT(v.vehicle_id) AS vehicles
FROM Customer c
JOIN Vehicle v ON v.person_id = c.person_id
GROUP BY c.person_id, c.name
HAVING COUNT(v.vehicle_id) > 1;
```

Cypher

```cypher
MATCH (c:Customer)-[:OWNS]->(v:Vehicle)
WITH c, count(v) AS vehicles
WHERE vehicles > 1
RETURN c.name, vehicles;
```

SQL requires `GROUP BY` and `HAVING` to filter on an aggregate. Cypher groups by the remaining variables in the `WITH` clause, after which an ordinary `WHERE` filters on the count. The two are of similar length and neither is notably harder.

Example 2. Customers affected by an Oil Filter recall (G4)

SQL

```sql
SELECT DISTINCT c.name, c.phone, v.registration_no
FROM SparePart p
JOIN ServiceItem si ON si.part_id    = p.part_id
JOIN ServiceCard sc ON sc.service_id = si.service_id
JOIN Vehicle     v  ON v.vehicle_id  = sc.vehicle_id
JOIN Customer    c  ON c.person_id   = v.person_id
WHERE p.part_name = 'Oil Filter';
```

Cypher

```cypher
MATCH (p:SparePart {part_name:'Oil Filter'})<-[:USES]-
      (:PartUsed)<-[:INCLUDES]-(:ServiceCard)<-[:HAS_SERVICE]-
      (v:Vehicle)<-[:OWNS]-(c:Customer)
RETURN DISTINCT c.name, c.phone, v.registration_no;
```

The SQL version uses four joins, each with its own `ON` condition, whereas the Cypher version uses one pattern that follows the path. The Cypher reads in the same order as the question (part, line item, service card, vehicle, owner), and an additional hop is one more step in the pattern rather than another join.

Example 3. Revenue per mechanic (G8)

SQL

```sql
SELECT m.name, COUNT(i.invoice_id) AS invoices, SUM(i.total) AS revenue
FROM Mechanic m
JOIN ServiceCard sc ON sc.mechanic_id = m.mechanic_id
JOIN Invoice     i  ON i.service_id   = sc.service_id
GROUP BY m.mechanic_id, m.name
ORDER BY revenue DESC;
```

Cypher

```cypher
MATCH (m:Mechanic)-[:ASSIGNED_TO]->(:ServiceCard)-[:GENERATES]->(i:Invoice)
RETURN m.name, count(i) AS invoices, sum(i.total) AS revenue
ORDER BY revenue DESC;
```

This is the closest of the three examples. Both versions are short, and MySQL handles this kind of aggregation comfortably. It is included to show that the difference is small for report-style queries.

One query has no concise SQL equivalent: G7, the shortest path from a customer to a supplier. In MySQL it would require a recursive common table expression in which the tables to be joined at each step are specified explicitly. In Cypher it is a single call, because the relationships are stored in the database.

---

14. Conclusion

The Vehicle Service Management System designed in DA1 and implemented in MySQL in DA2 was successfully re-implemented as a graph in Neo4j. Customers, vehicles, service

cards, mechanics, service items, parts, suppliers, invoices and payments were modelled as nodes and relationships, and the ten business queries were translated into Cypher without difficulty. Nine graph queries and several database features extended the work beyond what the relational version covered.

The benefit of the graph model was not uniform. Simple lookups and aggregations (Q1 to Q10 and G8) were expressed with comparable effort in Cypher and SQL. The difference became clear in queries that follow a chain of relationships, namely the recall list in G4, the supplier impact analysis in G3 and the shortest path in G7. The use of labels for subtypes, and of a single relationship for the part–supplier association, also produced a smaller model than the MySQL schema.

The limitations were equally evident. Neo4j provides no foreign keys or triggers, so the audit log and the stock check must be written into the queries, and the application must apply them consistently. The stock update in D1 does nothing, without reporting an error, when stock is insufficient, and this would need to be corrected before production use.

For an actual service centre, a reasonable arrangement would be to retain billing and routine reporting in MySQL and to use a graph database alongside it for relationship-centred questions. The principal conclusion of this assignment is that the choice of database should be determined by the questions the system is expected to answer most often.

---

15. References

1. Neo4j Documentation. *Cypher Manual.* https://neo4j.com/docs/cypher-manual/current/
2. Neo4j Documentation. *Cypher Manual: Constraints.* https://neo4j.com/docs/cypher-manual/current/constraints
3. Neo4j Documentation. *Cypher Manual: Indexes* (range indexes and full-text indexes). https://neo4j.com/docs/cypher-manual/current/indexes/
4. Neo4j Documentation. *Neo4j AuraDB.* https://neo4j.com/docs/aura/
5. Neo4j Documentation. *Operations Manual: Transaction management.* https://neo4j.com/docs/operations-manual/current/
6. Project repository, *DBMS-DA3-Neo4j-Vehicle-Service.* https://github.com/NandithaDanda/DBMS-DA3-Neo4j-Vehicle-Service